Executable Self-Justifying Axiom Systems in Proflog

LOPSTR+PPDP 2026 System Description

Author information pending

Affiliation pending

LOPSTR+PPDP 2026

System link: `https://github.com/autarkenterprises/proflog`

Core Claim

System-description thesis

Willard-style self-justifying axiom systems can be made executable in Proflog by treating axiom bases, formula/proof codes, proof predicates, and generated self-consistency sentences as ordinary first-order objects.

- Proflog supplies the proof-producing tableau kernel.
- SJAS supplies the weak arithmetic and restricted reflection target.
- The implementation exposes the programming-language boundary: reflection changes what the internal proof predicate may cite.

Why This Boundary Matters

Arithmetic kept

- constants 0, 1
- successor-like binary numerals
- total addition
- enough coding to inspect formulas and proofs

Arithmetic withheld

- no total multiplication symbol
- multiplication appears as $mult(x, y, z)$
- proof/code manipulation must respect that restriction

The implementation makes the restriction operational rather than only metamathematical.

Why Proflog Is a Useful Host

- Proflog programs are first-order formulae with equality.
- Execution uses semantic tableau closure plus a Procedure Call rule.
- The same kernel can execute user programs and check encoded proof certificates.

Key alignment

The deduction method D for the generated $IS\#_D(\beta)$ system is the ordinary Proflog first-order/equality tableau kernel.

Implemented Profile

Inputs	Generated	Compiled	Queried
profile choice	Group-Zero/1	executable Proflog program	direct calls
finite β	Group-2 beta	`axiom-member` system/formula/proof codes	SJAS theorem calls
reflected clauses	Group-2b reflected		certificate checks
external clauses	Group-3 SelfCons		

Authoring Boundary

Reflected clause

```
demo(x) :- x = 1.
```

Executable by Proflog and included as a Group-2b axiom citeable by the internal proof predicate.

External clause

```
app(x) :- x = 1.
```

Executable by ordinary procedure call, but absent from the system code and generated self-consistency claim.

Codes Are Inspectable

- First prototype: finite labels for formulas and proof targets.
- Current compact form: byte/base-64 code terms.
- U-grounding form: public codes are binary numerals over the SJAS arithmetic vocabulary.

```
(code-N b0 ... bN-1)
```

```
0, 1, (dbl n), (add n m)
```

The promoted relations consume codes; proof validity is not a host-side lookup table.

Proof-Predicate Path

Decode formula	Substitute	Check proof

`wff`
formula classes
`neg-pair`
	`subst-code/2`
	`subst-prf/4`
	Level-1 fixed point
		`tableau-proof/3`
		decode certificate
		invoke Proflog kernel

A certificate succeeds only when the decoded proof closes the decoded target from the active generated system.

Example: Beta Axiom or Procedure

Beta axiom

```
forall x. mult(2, 2, x) -> composite(x)
```

`composite(4)` is an SJAS theorem, but there is no runtime `composite/1` procedure.

Reflected procedure

```
composite(x) :- mult(2, 2, x).
```

It is executable, citeable as Group-2b, and supports answer mode for `composite(x)`.

Evaluation Evidence

Focused gate

```
lein test-proflog-sjas
```

35 tests
221 assertions
0 failures, 0 errors

Slow SJAS selector

```
lein test-proflog-sjas-slow
```

5 tests
22 assertions
0 failures, 0 errors

Coverage includes reflected/external boundaries, U-grounding arithmetic, answer mode, partial synthesis, structural decoding, and invalid-certificate rejection.

What This Does Not Claim

Limits of the current system

- finite ordinary-tableau $IS\#_D(\beta)$ substrate, not every Willard variant;
- no Tab-1 theorem reuse or proof-list checker yet;
- open proof-code synthesis remains operationally expensive;
- consistency remains Willard's mathematical metatheorem, not a machine-checked proof in this implementation.

Submission Fit

- Working system: public Proflog repository and focused examples.
- Novel aspect: executable SJAS profile inside a tableau logic programming system.
- Declarative-programming angle: reflection is an authoring choice that changes proof-citation power.
- Verification angle: object-language proof predicates route to real kernel certificate checks.

Takeaway

Executable reflection with a visible cost model

Proflog makes the SJAS construction concrete enough to program against: generated finite axiom systems, arithmetized syntax, kernel-checked proof predicates, and a clear boundary between reflected and external code.

The next research step is to push more syntax construction and proof-code manipulation into the restricted U-grounding arithmetic so programs directly experience the absence of total multiplication.